

Chapter 8. Collection API enhancements

This chapter covers

- Using collection factories
- Learning new idiomatic patterns to use with `List` and `Set`
- Learning idiomatic patterns to work with `Map`

Your life as a Java developer would be rather lonely without the Collection API. Collections are used in every Java applications. In previous chapters, you saw how useful the combination of Collections with the Streams API is for expressing data processing queries. Nonetheless, the Collection API had various deficiencies, which made it verbose and error-prone to use at times.

In this chapter, you will learn about new additions to the Collection API in Java 8 and Java 9 that will make your life easier. First, you learn about the collections factories in Java 9—new additions that simplify the process of creating small lists, sets, and maps. Next, you learn how to apply idiomatic removal and replacement patterns in lists and sets thanks to Java 8 enhancements. Finally, you learn about new convenience operations that are available to work with maps.

Chapter 9 explores a wider range of techniques for refactoring old-style Java code.

8.1. Collection factories

Java 9 introduced a few convenient ways to create small collection objects. First, we'll review why programmers needed a better way to do things; then we'll show you how to use the new factory methods.

How would you create a small list of elements in Java? You might want to group the names of your friends who are going on a holiday, for example.

Other formats available



Audiobook



```
friends = new ArrayList<>();
friends.add("Raphael");
friends.add("Olivia");
friends.add("Thibaut");
```

But that's quite a few lines to write for storing three strings! A more convenient way to write this code is to use the `Arrays.asList()` factory method:

```
List<String> friends
    = Arrays.asList("Raphael", "Olivia", "Thibaut");
```

You get a fixed-sized list that you can update, but not add elements to or remove elements from. Attempting to add elements, for example, results in an `UnsupportedOperationException`, but updating by using the method `set` is allowed:

```
List<String> friends = Arrays.asList("Raphael", "Olivia");
friends.set(0, "Richard");
friends.add("Thibaut"); 1
```

- **1 throws an `UnsupportedOperationException`**

This behavior seems slightly surprising because the underlying list is backed by a mutable array of fixed size.

How about a `Set`? Unfortunately, there's no `Arrays.asSet()` factory method, so you need another trick. You can use the `HashSet` constructor, which accepts a `List`:

```
Set<String> friends =
    new HashSet<>(Arrays.asList("Raphael", "Olivia", "Thibaut"));
```

Alternatively you could use the Streams API:

```
Set<String> friends
    = Stream.of("Raphael", "Olivia", "Thibaut")
        .collect(Collectors.toSet());
```

Both solutions, however, are far from elegant and involve unnecessary object allocations behind the scenes. Also note that you get a mutable `Set`

Other formats available



`Map` is an elegant way of creating small maps, but don't worry; Java 9 added factory methods to make your life simpler when you need to create small lists, sets, and maps.

Collection literals

Some languages, including Python and Groovy, support collection literals, which let you create collections by using special syntax, such as `[42, 1, 5]` to create a list of three numbers. Java doesn't provide syntactic support because language changes come with a high maintenance cost and restrict future use of a possible syntax. Instead, Java 9 adds support by enhancing the Collection API.

We begin the tour of new ways of creating collections in Java by showing you what's new with Lists.

8.1.1. List factory

You can create a list simply by calling the factory method `List.of`:

```
List<String> friends = List.of("Raphael", "Olivia", "Thibaut");  
System.out.println(friends);
```

1

- 1 [Raphael, Olivia, Thibaut]

You'll notice something strange, however. Try to add an element to your list of friends:

```
List<String> friends = List.of("Raphael", "Olivia", "Thibaut");  
friends.add("Chih-Chun");
```

Running this code results in a `java.lang.UnsupportedOperationException`. In fact, the list that's produced is immutable. Replacing an item with the `set()` method throws a similar exception. You won't be able to modify it by using the `set` method either. This restriction is a good thing, however, as it protects you from unwanted mutations of the collections. Nothing is stopping you from having elements that are mutable themselves. If you need a mutable list, you can still instantiate one manually. Finally, note that to prevent unexpected bugs and enable a more-compact internal representation, null elements are disallowed.

Other formats available

 Audiobook 

If you further inspect the `List` interface, you notice several overloaded variants of `List.of`:

```
static <E> List<E> of(E e1, E e2, E e3, E e4)
static <E> List<E> of(E e1, E e2, E e3, E e4, E e5)
```

You may wonder why the Java API didn't have one method that uses varargs to accept an arbitrary number of elements in the following style:

```
static <E> List<E> of(E... elements)
```

Under the hood, the varargs version allocates an extra array, which is wrapped up inside a list. You pay the cost for allocating an array, initializing it, and having it garbage-collected later. By providing a fixed number of elements (up to ten) through an API, you don't pay this cost. Note that you can still create `List.of` using more than ten elements, but in that case the varargs signature is invoked. You also see this pattern appearing with `Set.of` and `Map.of`.

You may wonder whether you should use the Streams API instead of the new collection factory methods to create such lists. After all, you saw in previous chapters that you can use the `Collectors.toList()` collector to transform a stream into a list. Unless you need to set up some form of data processing and transformation of the data, we recommend that you use the factory methods; they're simpler to use, and the implementation of the factory methods is simpler and more adequate.

Now that you've learned about a new factory method for `List`, in the next section, you will work with `Sets`.

8.1.2. Set factory

As with `List.of`, you can create an immutable `Set` out of a list of elements:

```
Set<String> friends = Set.of("Raphael", "Olivia", "Thibaut");
System.out.println(friends);
```

1

Other formats available



Providing a duplicated element, you receive an `IllegalArgumentException`. This exception reflects the contract that sets enforce uniqueness of the elements they contain:

```
Set<String> friends = Set.of("Raphael", "Olivia", "Olivia");
```

1

- **1 java.lang.IllegalArgumentException: duplicate element: Olivia**

Another popular data structure in Java is Map. In the next section, you learn about new ways of creating Maps.

8.1.3. Map factories

Creating a map is a bit more complicated than creating lists and sets because you have to include both the key and the value. You have two ways to initialize an immutable map in Java 9. You can use the factory method `Map.of`, which alternates between keys and values:

```
Map<String, Integer> ageOfFriends
    = Map.of("Raphael", 30, "Olivia", 25, "Thibaut", 26);
System.out.println(ageOfFriends);
```

1

- **1 {Olivia=25, Raphael=30, Thibaut=26}**

This method is convenient if you want to create a small map of up to ten keys and values. To go beyond this, use the alternative factory method called `Map.ofEntries`, which takes `Map.Entry<K, V>` objects but is implemented with varargs. This method requires additional object allocations to wrap up a key and a value:

```
import static java.util.Map.entry;
Map<String, Integer> ageOfFriends
    = Map.ofEntries(entry("Raphael", 30),
                    entry("Olivia", 25),
                    entry("Thibaut", 26));
System.out.println(ageOfFriends);
```

1

- **1 {Olivia=25, Raphael=30, Thibaut=26}**

`Map.entry` is a new factory method to create `Map.Entry` objects.

Other formats available



out of the following snippet?

```
List<String> actors = List.of("Keanu", "Jessica")
actors.set(0, "Brad");
```

```
System.out.println(actors)
```

Answer:

An `UnsupportedOperationException` is thrown. The collection produced by `List.of` is immutable.

So far, you've seen that the new Java 9 factory methods allow you to create collections more simply. But in practice, you have to process the collections. In the next section, you learn about a few new enhancements to `List` and `Set` that implement common processing patterns out of the box.

8.2. Working with List and Set

Java 8 introduced a couple of methods into the `List` and `Set` interfaces:

- `removeIf` removes element matching a predicate. It's available on all classes that implement `List` or `Set` (and is inherited from the `Collection` interface).
- `replaceAll` is available on `List` and replaces elements using a `(UnaryOperator)` function.
- `sort` is also available on the `List` interface and sorts the list itself.

All these methods mutate the collections on which they're invoked. In other words, they change the collection itself, unlike stream operations, which produce a new (copied) result. Why would such methods be added? Modifying collections can be error-prone and verbose. So Java 8 added `removeIf` and `replaceAll` to help.

8.2.1. removeIf

Consider the following code, which tries to remove transactions that have a reference code starting with a digit:

```
for (Transaction transaction : transactions) {  
    if (transaction.getReferenceCode().charAt(0)) {  
        remove(transaction);  
    }  
}
```

Other formats available



Can you see the problem? Unfortunately, this code may result in a `ConcurrentModificationException`. Why? Under the hood, the `for-`

each loop uses an `Iterator` object, so the code executed is as follows:

```
for (Iterator<Transaction> iterator = transactions.iterator();
     iterator.hasNext(); ) {
    Transaction transaction = iterator.next();
    if(Character.isDigit(transaction.getReferenceCode().charAt(0))) {
        transactions.remove(transaction);
    }
}
```

- **1 Problem we are iterating and modifying the collection through two separate objects**

Notice that two separate objects manage the collection:

- The `Iterator` object, which is querying the source by using `next()` and `hasNext()`
- The `Collection` object itself, which is removing the element by calling `remove()`

As a result, the state of the iterator is no longer synced with the state of the collection, and vice versa. To solve this problem, you have to use the `Iterator` object explicitly and call its `remove()` method:

```
for (Iterator<Transaction> iterator = transactions.iterator();
     iterator.hasNext(); ) {
    Transaction transaction = iterator.next();
    if(Character.isDigit(transaction.getReferenceCode().charAt(0))) {
        iterator.remove();
    }
}
```

This code has become fairly verbose to write. This code pattern is now directly expressible with the Java 8 `removeIf` method, which is not only simpler but also protects you from these bugs. It takes a predicate indicating which elements to remove:

```
transactions.removeIf(transaction ->
    !Character.isDigit(transaction.getReferenceCode().charAt(0)));
```

Other formats available



Audiobook

removing an element, you want to replace

it. For this purpose, Java 8 added `replaceAll`.

8.2.2. `replaceAll`

The `replaceAll` method on the `List` interface lets you replace each element in a list with a new one. Using the Streams API, you could solve this problem as follows:

```
referenceCodes.stream()
    .map(code -> Character.toUpperCase(code.charAt(0)) +
        code.substring(1))
    .collect(Collectors.toList())
    .forEach(System.out::println);
```

- **1 [a12, C14, b13]**
- **2 outputs A12, C14, B13**

This code results in a new collection of strings, however. You want a way to update the existing collection. You can use a `ListIterator` object as follows (supporting a `set()` method to replace an element):

```
for (ListIterator<String> iterator = referenceCodes.listIterator();
     iterator.hasNext(); ) {
    String code = iterator.next();
    iterator.set(Character.toUpperCase(code.charAt(0)) + code.substring(1));
}
```

As you can see, this code is fairly verbose. In addition, as we explained earlier, using `Iterator` objects in conjunction with collection objects can be error-prone by mixing iteration and modification of the collection. In Java 8, you can simply write

```
referenceCodes.replaceAll(code -> Character.toUpperCase(code.charAt(0)) +
    code.substring(1));
```

You've learned what's new with `List` and `Set`, but don't forget about `Map`. New additions to the `Map` interface are covered in the next section.

8.3. Working with Map

Java 8 introduced several default methods supported by the `Map` inter-

Other formats available



Audiobook



ferred in detail in [chapter 13](#), but here you'll see how they're implemented (as implemented methods in an interface.) The reason for this is to help you write more concise code

by using a readily available idiomatic pattern instead of implementing it yourself. We look at these operations in the following sections, starting with the shiny new `forEach`.

8.3.1. forEach

Iterating over the keys and values of a Map has traditionally been awkward. In fact, you needed to use an iterator of a `Map.Entry<K, V>` over the entry set of a Map:

```
for(Map.Entry<String, Integer> entry: ageOfFriends.entrySet()) {
    String friend = entry.getKey();
    Integer age = entry.getValue();
    System.out.println(friend + " is " + age + " years old");
}
```

Since Java 8, the Map interface has supported the `forEach` method, which accepts a `BiConsumer`, taking the key and value as arguments. Using `forEach` makes your code more concise:

```
ageOfFriends.forEach((friend, age) -> System.out.println(friend + " is
    age + " years old"));
```

A concern related to iterating over data is sorting it. Java 8 introduced a couple of convenient ways to compare entries in a Map.

8.3.2. Sorting

Two new utilities let you sort the entries of a map by values or keys:

- `Entry.comparingByValue`
- `Entry.comparingByKey`

The code

```
Map<String, String> favouriteMovies
    = Map.ofEntries(entry("Raphael", "Star Wars"),
        entry("Cristina", "Matrix"),
        entry("Olivia",
            "James Bond"));
```

```
favouriteMovies
```

Other formats available



```
ringByKey())
tem.out::println);
```

1

- **1 Processes the elements of the stream in alphabetic order based on the person's name**

outputs, in order:

```
Cristina=Matrix
Olivia=James Bond
Raphael=Star Wars
```

HashMap and Performance

The internal structure of a HashMap was updated in Java 8 to improve performance. Entries of a map typically are stored in buckets accessed by the generated hashCode of the key. But if many keys return the same hashCode, performance deteriorates because buckets are implemented as LinkedLists with $O(n)$ retrieval. Nowadays, when the buckets become too big, they're replaced dynamically with sorted trees, which have $O(\log(n))$ retrieval and improve the lookup of colliding elements. Note that this use of sorted trees is possible only when the keys are Comparable (such as String or Number classes).

Another common pattern is how to act when the key you're looking up in the Map isn't present. The new `getOrDefault` method can help.

8.3.3. getOrDefault

When the key you're looking up isn't present, you receive a null reference that you have to check against to prevent a `NullPointerException`. A common design style is to provide a default value instead. Now you can encode this idea more simply by using the `getOrDefault` method. This method takes the key as the first argument and a default value (to be used when the key is absent from the Map) as the second argument:

```
Map<String, String> favouriteMovies
    = Map.ofEntries(entry("Raphael", "Star Wars"),
        entry("Olivia", "James Bond"));

System.out.println(favouriteMovies.getOrDefault("Olivia", "Matrix"));
System.out.println(favouriteMovies.getOrDefault("Thibaut", "Matrix"));
```

Other formats available

 Audiobook 

- 2 Outputs Matrix

Note that if the key existed in the Map but was accidentally associated with a null value, `getOrDefault` can still return null. Also note that the

expression you pass as a fallback is always evaluated, whether the key exists or not.

Java 8 also included a few more advanced patterns to deal with the presence and absence of values for a given key. You will learn about these new methods in the next section.

8.3.4. Compute patterns

Sometimes, you want to perform an operation conditionally and store its result, depending on whether a key is present or absent in a Map. You may want to cache the result of an expensive operation given a key, for example. If the key is present, there's no need to recalculate the result. Three new operations can help:

- `computeIfAbsent`—If there's no specified value for the given key (it's absent or its value is null), calculate a new value by using the key and add it to the Map.
- `computeIfPresent`—If the specified key is present, calculate a new value for it and add it to the Map.
- `compute`—This operation calculates a new value for a given key and stores it in the Map.

One use of `computeIfAbsent` is for caching information. Suppose that you parse each line of a set of files and calculate their SHA-256 representation. If you've processed the data previously, there's no need to recalculate it.

Now suppose that you implement a cache by using a Map, and you use an instance of `MessageDigest` to calculate SHA-256 hashes:

```
Map<String, byte[]> dataToHash = new HashMap<>();
MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
```

Then you can iterate through the data and cache the results:

```
lines.forEach(line ->
    dataToHash.computeIfAbsent(line,
                                this::calculateDigest));

private byte[] calculateDigest(String key) {
    return messageDigest.digest(key.getBytes(StandardCharsets.UTF_8));
}
```

Other formats available



Audiobook



- 1 line is the key to look up in the map.

- **2 The operation to execute if the key is absent**
- **3 The helper that will calculate a hash for the given key**

This pattern is also useful for conveniently dealing with maps that store multiple values. If you need to add an element to a `Map<K, List<V>>`, you need to ensure that the entry has been initialized. This pattern is a verbose one to put in place. Suppose that you want to build up a list of movies for your friend Raphael:

```
String friend = "Raphael";
List<String> movies = friendsToMovies.get(friend);
if(movies == null) { 1
    movies = new ArrayList<>();
    friendsToMovies.put(friend, movies);
}
movies.add("Star Wars"); 2

System.out.println(friendsToMovies); 3
```

- **1 Check that the list was initialized.**
- **2 Add the movie.**
- **3 {Raphael: [Star Wars]}**

How can you use `computeIfAbsent` instead? It returns the calculated value after adding it to the Map if the key wasn't found; otherwise, it returns the existing value. You can use it as follows:

```
friendsToMovies.computeIfAbsent("Raphael", name -> new ArrayList<>())
    .add("Star Wars");
```

- **1 {Raphael: [Star Wars]}**

The `computeIfPresent` method calculates a new value if the current value associated with the key is present in the Map and non-null. Note a subtlety: if the function that produces the value returns null, the current mapping is removed from the Map. If you need to remove a mapping, however, an overloaded version of the `remove` method is better suited to the task. You learn about this method in the next section.

Other formats available



You already know about the `remove` method that lets you remove a Map entry for a given key. Since Java 8, an overloaded version removes an entry only if the key is associated with a specific value. Previously, this code

is how you'd implement this behavior (we have nothing against Tom Cruise, but *Jack Reacher 2* received poor reviews):

```
String key = "Raphael";
String value = "Jack Reacher 2";
if (favouriteMovies.containsKey(key) &&
    Objects.equals(favouriteMovies.get(key), value)) {
    favouriteMovies.remove(key);
    return true;
}
else {
    return false;
}
```

Here is how you can do the same thing now, which you have to admit is much more to the point:

```
favouriteMovies.remove(key, value);
```

In the next section, you learn about ways of replacing elements in and removing elements from a Map.

8.3.6. Replacement patterns

Map has two new methods that let you replace the entries inside a Map:

- **replaceAll**—Replaces each entry's value with the result of applying a `BiFunction`. This method works similarly to `replaceAll` on a `List`, which you saw earlier.
- **Replace**—Lets you replace a value in the Map if a key is present. An additional overload replaces the value only if it the key is mapped to a certain value.

You could format all the values in a Map as follows:

```
Map<String, String> favouriteMovies = new HashMap<>();
favouriteMovies.put("Raphael", "Star Wars");
favouriteMovies.put("Olivia", "james bond");
favouriteMovies.replaceAll((friend, movie) -> movie.toUpperCase(),
    favouriteMovies);
```

Other formats available



- 1 We have to use a mutable map since we will be using `replaceAll`
- 2 {Olivia=JAMES BOND, Raphael=STAR WARS}

The replace patterns you've learned work with a single Map. But what if you have to combine and replace values from two Maps? You can use a new merge method for that task.

8.3.7. Merge

Suppose that you'd like to merge two intermediate Maps, perhaps two separate Maps for two groups of contacts. You can use `putAll` as follows:

```
Map<String, String> family = Map.ofEntries(
    entry("Teo", "Star Wars"), entry("Cristina", "James Bond"));
Map<String, String> friends = Map.ofEntries(
    entry("Raphael", "Star Wars"));
Map<String, String> everyone = new HashMap<>(family);
everyone.putAll(friends); 1
System.out.println(everyone); 2
```

- **1 Copies all the entries from friends into everyone**
- **2 {Cristina=James Bond, Raphael=Star Wars, Teo=Star Wars}**

This code works as expected as long as you don't have duplicate keys. If you require more flexibility in how values are combined, you can use the new merge method. This method takes a `BiFunction` to merge values that have a duplicate key. Suppose that Cristina is in both the family and friends maps but with different associated movies:

```
Map<String, String> family = Map.ofEntries(
    entry("Teo", "Star Wars"), entry("Cristina", "James Bond"));
Map<String, String> friends = Map.ofEntries(
    entry("Raphael", "Star Wars"), entry("Cristina", "Matrix"));
```

Then you could use the merge method in combination with `forEach` to provide a way to deal with the conflict. The following code concatenates the string names of the two movies:

```
Map<String, String> everyone = new HashMap<>(family);
friends.forEach((k, v) ->
    everyone.merge(k, v, (movie1, movie2) -> movie1 + " & " + movie2));
```

Other formats available



concatenates the two values

- **2 Outputs {Raphael=Star Wars, Cristina=James Bond & Matrix, Teo=Star Wars}**

Note that the merge method has a fairly complex way to deal with nulls, as specified in the Javadoc:

If the specified key is not already associated with a value or is associated with null, [merge] associates it with the given non-null value. Otherwise, [merge] replaces the associated value with the [result] of the given remapping function, or removes [it] if the result is null.

You can also use merge to implement initialization checks. Suppose that you have a Map for recording how many times a movie is watched. You need to check that the key representing the movie is in the map before you can increment its value:

```
Map<String, Long> moviesToCount = new HashMap<>();
String movieName = "JamesBond";
long count = moviesToCount.get(movieName);
if(count == null) {
    moviesToCount.put(movieName, 1);
}
else {
    moviesToCount.put(movieName, count + 1);
}
```

This code can be rewritten as

```
moviesToCount.merge(movieName, 1L, (key, count) -> count + 1L);
```

The second argument to merge in this case is 1L. The Javadoc specifies that this argument is “the non-null value to be merged with the existing value associated with the key or, if no existing value or a null value is associated with the key, to be associated with the key.” Because the value returned for that key is null, the value 1 is provided the first time around. The next time around, because the value for the key was initialized to the value of 1, the BiFunction is applied to increment the count.

Quiz 8.2

Other formats available



code does, and think of what idiomatic op-
tify what it's doing:

```
Map<String, Integer> movies = new HashMap<>();
movies.put("JamesBond", 20);
movies.put("Matrix", 15);
movies.put("Harry Potter", 5);
```

```

Iterator<Map.Entry<String, Integer>> iterator =
    movies.entrySet().iterator();
while(iterator.hasNext()) {
    Map.Entry<String, Integer> entry = iterator.next();
    if(entry.getValue() < 10) {
        iterator.remove();
    }
}
System.out.println(movies);

```

1

- **1 {Matrix=15, JamesBond=20}**

Answer:

You can use the `removeIf` method on the map's entry set, which takes a predicate and removes the elements:

```

movies.entrySet().removeIf(entry -> entry.getValue() < 10);

```

You've learned about the additions to the `Map` interface. New enhancements were added to a cousin: `ConcurrentHashMap` which you will learn about next.

8.4. Improved ConcurrentHashMap

The `ConcurrentHashMap` class was introduced to provide a more modern `HashMap`, which is also concurrency friendly. `ConcurrentHashMap` allows concurrent add and update operations that lock only certain parts of the internal data structure. Thus, read and write operations have improved performance compared with the synchronized `Hashtable` alternative. (Note that the standard `HashMap` is unsynchronized.)

8.4.1. Reduce and Search

`ConcurrentHashMap` supports three new kinds of operations, reminiscent of what you saw with streams:

Other formats available



Audiobook



an action for each (key, value)

, value) given a reduction function into a

- **search**—Applies a function on each (key, value) until the function produces a non-null result

Each kind of operation supports four forms, accepting functions with keys, values, `Map.Entry`, and (key, value) arguments:

- Operates with keys and values (`forEach`, `reduce`, `search`)
- Operates with keys (`forEachKey`, `reduceKeys`, `searchKeys`)
- Operates with values (`forEachValue`, `reduceValues`, `searchValues`)
- Operates with `Map.Entry` objects (`forEachEntry`, `reduceEntries`, `searchEntries`)

Note that these operations don't lock the state of the `ConcurrentHashMap`; they operate on the elements as they go along. The functions supplied to these operations shouldn't depend on any ordering or on any other objects or values that may change while computation is in progress.

In addition, you need to specify a parallelism threshold for all these operations. The operations execute sequentially if the current size of the map is less than the given threshold. A value of 1 enables maximal parallelism using the common thread pool. A threshold value of `Long.MAX_VALUE` runs the operation on a single thread. You generally should stick to these values unless your software architecture has advanced resource-use optimization.

In this example, you use the `reduceValues` method to find the maximum value in the map:

```
ConcurrentHashMap<String, Long> map = new ConcurrentHashMap<>();
long parallelismThreshold = 1;
Optional<Integer> maxValue =
    Optional.ofNullable(map.reduceValues(parallelismThreshold, Long::ma
```

- **1 A `ConcurrentHashMap`, presumed to be updated to contain several keys and values**

Note the primitive specializations for `int`, `long`, and `double` for each reduce operation (`reduceValuesToInt`, `reduceKeysToLong`, and so on), which are more efficient as they prevent boxing.

Other formats available



The `ConcurrentHashMap` class provides a new method called `mappingCount`, which returns the number of mappings in the map as a `long`. You should use it for new code in preference to the `size` method, which re-

turns an `int`. Doing so future proofs your code for use when the number of mappings no longer fits in an `int`.

8.4.3. Set views

The `ConcurrentHashMap` class provides a new method called `keySet` that returns a view of the `ConcurrentHashMap` as a `Set`. (Changes in the map are reflected in the `Set`, and vice versa.) You can also create a `Set` backed by a `ConcurrentHashMap` by using the new static method `newKeySet`.

Summary

- Java 9 supports collection factories, which let you create small immutable lists, sets, and maps by using `List.of`, `Set.of`, `Map.of`, and `Map.ofEntries`.
- The objects returned by these collection factories are immutable, which means that you can't change their state after creation.
- The `List` interface supports the default methods `removeIf`, `replaceAll`, and `sort`.
- The `Set` interface supports the default method `removeIf`.
- The `Map` interface includes several new default methods for common patterns and reduces the scope for bugs.
- `ConcurrentHashMap` supports the new default methods inherited from `Map` but provides thread-safe implementations for them.

Other formats available



Audiobook

